

UNIVERSIDAD EUROPEA DE VALENCIA
Grado en Física

Subject: Introduction to artificial intelligence

Activity 1 - Hooke's Law

Author:
Pau Gallego Cisteró

Date:
February 18, 2026

CONTENTS

1	Summary and objectives	2
2	Principles	2
2.1	Physical Basis: Hooke's Law	2
2.2	Stochastic Modeling and Gaussian Noise	2
2.3	Polynomial Regression and Model Complexity	2
2.4	Evaluation: Generalization and Overfitting	3
3	Graphs	3
4	Analysis	7
4.1	The Impact of Data Shuffling and Partitioning	7
4.2	False Security of Balanced Splits	7
4.3	Overfitting and High-Variance Models	8
5	Conclusions	8
6	Code	8

1. SUMMARY AND OBJECTIVES

This activity explores supervised machine learning by fitting polynomial regression models to noisy synthetic data generated via Hooke's Law. By partitioning the data into training and testing sets and comparing in-sample versus out-of-sample errors, the exercise demonstrates the relationship between model complexity, noise levels, and generalization, providing a practical framework for detecting and mitigating overfitting.

2. PRINCIPLES

The theoretical foundation of this activity is built upon two pillars: the physical modeling of a mechanical system and the statistical framework of supervised machine learning.

2.1. Physical Basis: Hooke's Law

Hooke's law is a principle of physics that states that the force (F) needed to extend or compress a spring by some distance (x) scales linearly with respect to that distance. In an ideal, one-dimensional system, this is expressed as

$$F = -kx; \tag{1}$$

where k is the spring constant, a positive real number representing the stiffness of the spring. In this exercise, k is set to 5. While the physical law is deterministic and linear, real-world measurements are subject to various sources of uncertainty.

2.2. Stochastic Modeling and Gaussian Noise

To simulate realistic experimental conditions, a noise term η is added to the deterministic physical model. We assume this noise follows a normal (Gaussian) distribution with a mean of zero and a variance σ^2

$$F = -kx + \eta, \quad \eta \sim \mathcal{N}(0, \sigma^2). \tag{2}$$

The standard deviation σ represents the "noise level." By varying σ between 0.5, 1, and 2, we can study how experimental "messiness" affects the ability of a machine learning algorithm to recover the underlying physical law.

2.3. Polynomial Regression and Model Complexity

In machine learning, the goal is to find a hypothesis function $h(x)$ that approximates the true underlying mapping. We use polynomial regression, where the model takes the form

$$h(x) = w_0 + w_1x + w_2x^2 + \cdots + w_dx^d = \sum_{i=0}^d w_ix^i. \tag{3}$$

where d is the degree of the polynomial.

- **Degree 1 (Linear):** Matches the actual physical law ($F \propto x$).
- **Degrees 2 and 3 (Quadratic and Cubic):** Introduce higher complexity. While these can fit the training data more closely, they may mistakenly model the random noise rather than the physical trend.

2.4. Evaluation: Generalization and Overfitting

The performance of the model is evaluated using the Mean Squared Error (MSE), defined as

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2. \quad (4)$$

To ensure the model has learned the physics and not just memorized the noise, we partition the data:

1. **Training Set:** Used to adjust the model parameters (w_i). The error calculated here is the In-sample error (E_{in}).
2. **Test Set:** Unseen data used to evaluate generalization. The error calculated here is the Out-of-sample error (E_{out}).

A significant discrepancy where E_{in} is low but E_{out} is high indicates overfitting, a phenomenon where the model over-optimizes for the specific noise in the training data, losing its predictive power for new observations.

3. GRAPHS

The figures presented below illustrate the systematic variations of the relationship under different experimental conditions. A comprehensive analysis and interpretation of these trends are provided in the subsequent Analysis section.

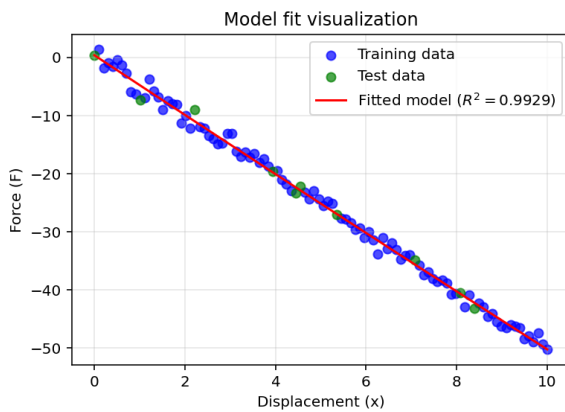


Figure 1: $N = 100$, train: 10%, $\sigma = 1$, degree = 2

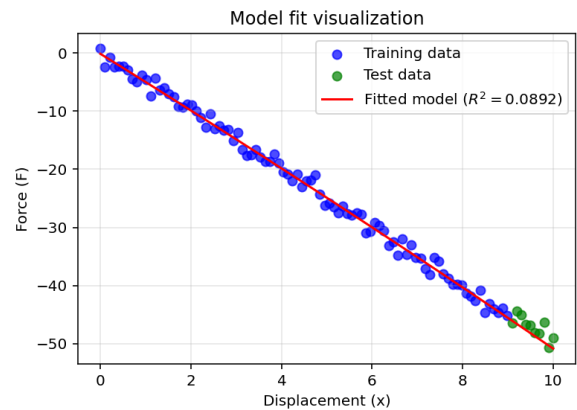


Figure 2: $N = 100$, train: 10%, $\sigma = 1$, degree = 2, separating test from train on the visualisation

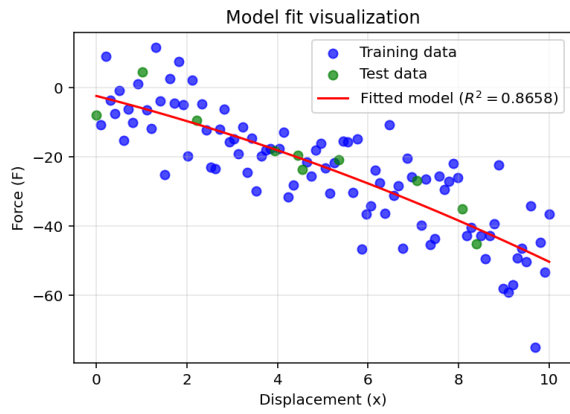


Figure 3: $N = 100$, 10%, $\sigma = 8$, degree = 2

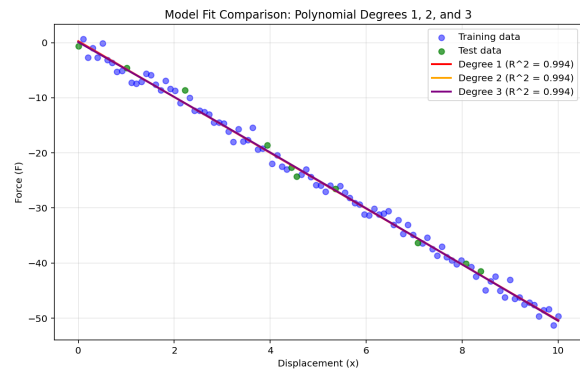


Figure 4: $N = 100$, 10%, $\sigma = 3$, 3 degrees in 1

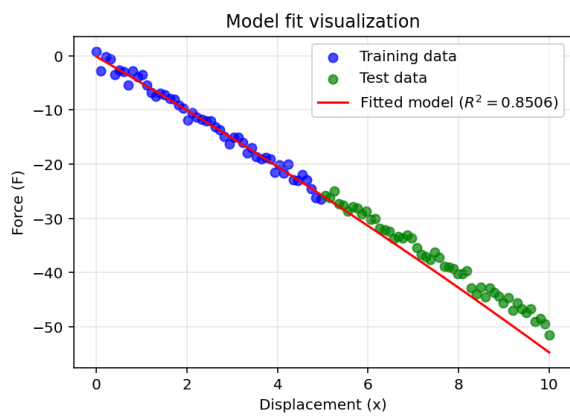


Figure 5: $N = 100$, 50%, $\sigma = 1$, degree = 2

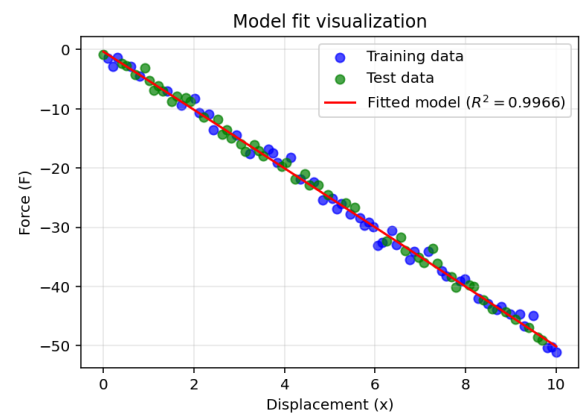


Figure 6: $N = 100$, 50%, $\sigma = 1$, degree = 2

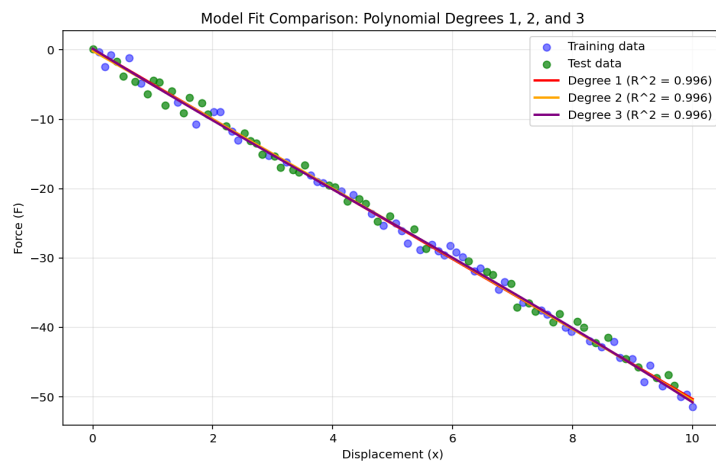


Figure 7: $N = 100$, 50%, $\sigma = 1$, 3 degrees in 1

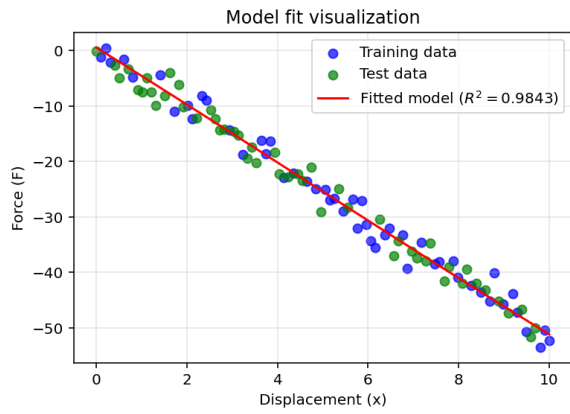


Figure 8: $N = 100$, 50%, $\sigma = 2$, degree = 3

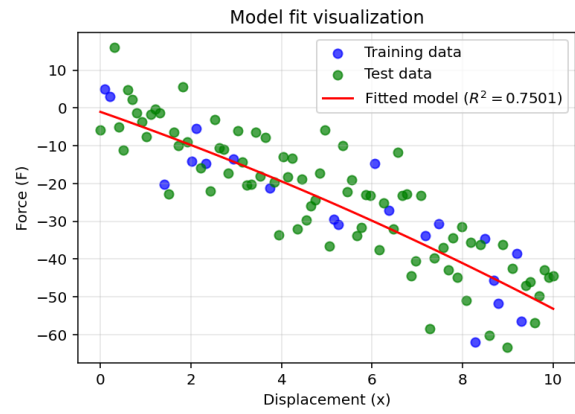


Figure 9: $N = 100$, 80%, $\sigma = 8$, degree = 2

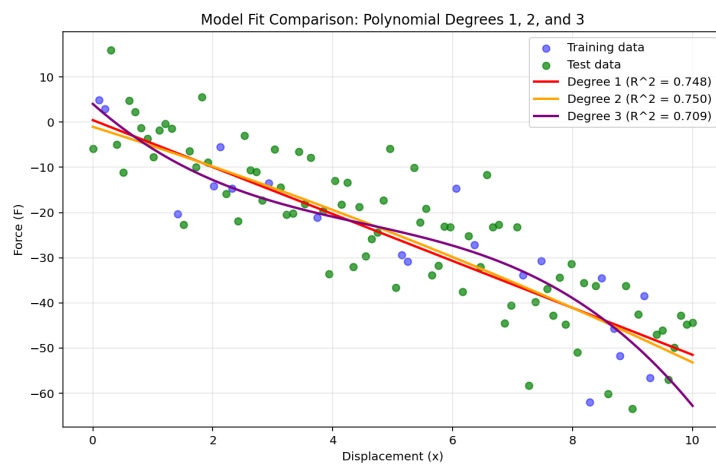


Figure 10: $N = 100$, 80%, $\sigma = 8$, 3 degrees in 1

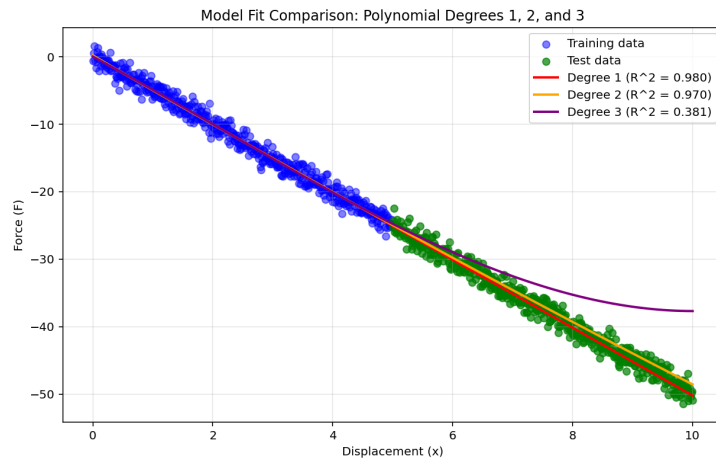


Figure 11: $N = 1000$, 50%, $\sigma = 1$, 3 degrees in 1

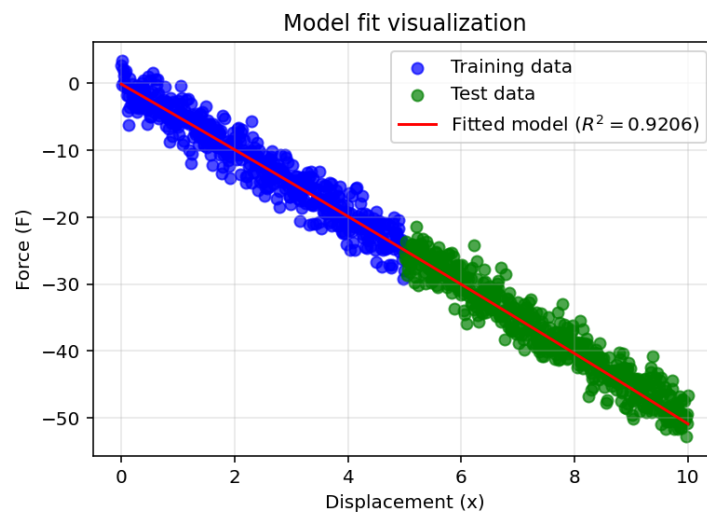


Figure 12: $N = 1000$, 50%, $\sigma = 2$, degree = 2

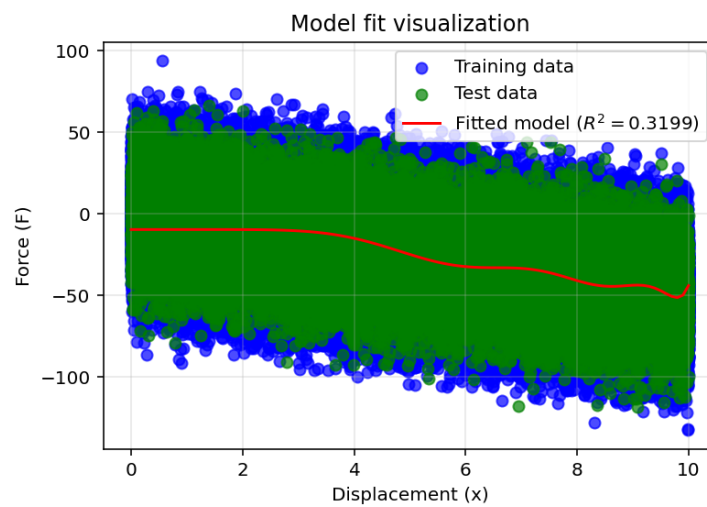


Figure 13: $N = 500,000$, 10%, $\sigma = 20$, degree = 15.

Source: Eduardo Varun Alcobilla

4. ANALYSIS

The following analysis evaluates the performance of polynomial regression models in recovering the underlying physical law, $F = -5x$, under varying experimental conditions. The effectiveness of these models is quantified through the Mean Squared Error (MSE) and the Coefficient of Determination (R^2), with a specific focus on the transition from well-fitted models to those suffering from overfitting.

Since Hooke's Law is linear, a degree 1 polynomial represents the most physically accurate hypothesis. However, increasing the polynomial degree allows the model to gain flexibility. The present analysis focuses heavily on the discrepancies introduced by data partitioning and model complexity.

4.1. The Impact of Data Shuffling and Partitioning

As observed in Figure 1, when training and testing data are shuffled and interleaved, the model yields a high $R^2 = 0.9929$. However, when the same framework is applied but the test set is separated to the end of the displacement range (Figure 2), the R^2 decreases dramatically to 0.0892; this implies that randomly shuffling the dataset is a critical step, without shuffling, the test set represents an out-of-distribution range where the model is attempting to predict trends in uncharted regions beyond its experience, rather than simply filling in the gaps within its known data range.

This issue is further highlighted in Figure 4 and Figure 11. In Figure 4, even with a small 10% training set, R^2 remains high (approx. 0.98) because the points are shuffled. Conversely, in Figure 11, we observe that as the degree increases to 3, the model begins to diverge at the boundaries, even with $N = 1000$, resulting in a significantly lower $R^2 = 0.381$ for the cubic fit.

4.2. False Security of Balanced Splits

When training and testing samples are distributed evenly (a 50/50% ratio), as shown in Figures 6 and 7, the R^2 remains consistently high (above 0.99). This occurs because the test points essentially become neighbors to the training points, reducing the challenge to simple interpolation.

Nevertheless, a high R^2 in this context does not necessarily imply superior predictive capability, by making the test set too statistically similar to the training set, we mask the model's inability to generalize to novel ranges. While the error appears lower, the model may simply be fitting a local distribution rather than learning the universal physical trend; Figures 9 and 10 show that when noise is high ($\sigma = 8$), a large training ratio (80%) still results in lower R^2 values (approx. 0.75) as the model struggles to distinguish the underlying physical law from the stochastic noise, leading to a significant increase in the out-of-sample error (E_{out}).

4.3. Overfitting and High-Variance Models

Signs of overfitting become particularly evident when both noise and polynomial degree are increased. Figure 13 provides an extreme case ($N = 500,000, \sigma = 20, d = 15$), where the regression algorithm prioritizes the minimization of residuals for noisy data points, this creates a characteristic “oscillation” at the boundaries resembling a tale (extrem model analysis encouraged by Eduardo).

This memorization of noise rather than the learning of the physical relationship is a high remarkable characteristic of overfitting, while a complex model may show a lower training error (E_{in}), it fails to remain robust when presented with experimental uncertainty; consequently, a simpler, lower-degree model is far more solid for representing Hooke’s Law across varying ranges.

5. CONCLUSIONS

After analyzing the model, we conclude that capturing the underlying physical law requires a careful balance between data quantity and noise reduction. Crucially, we found that the training and testing samples must be uniformly distributed across the displacement range. If the test data is isolated from the training distribution, the model fails to learn the global trend, resulting in a false representation of the system’s behavior.

It can also be noted that the degree 1 model exhibits high bias (it is rigid), but maintains low variance; in contrast, a Degree 15 model has low bias (it fits the data points closely) but suffers from enormous variance, where small changes in the dataset drastically alter the resulting curve; consequently, the success of Machine Learning lies in finding the optimal balance between these two properties, often being ruled by Ockham’s razor (simplicity is preferred).

Note: The initial R^2 inaccuracies occurred because the metric was improperly defined, unintentionally following correlation logic. Furthermore, disabling the shuffling process caused the numerator of the R^2 formula to fail in several cases; the model became numerically unstable, and while this behavior was physically nonsensical, it highlights the risks of extrapolation.

6. CODE

The following code is the one that was used to obtain the graphs on section "Experimental" making small variations to it as explained on previous sections aswell:

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import PolynomialFeatures
5 from sklearn.linear_model import LinearRegression
6 from sklearn.metrics import mean_squared_error
7
```

```
8 # Parameters
9 k = 5
10 sigma = 1
11 N = 100
12
13 # Generate data
14 x = np.linspace(0, 10, N)
15 noise = np.random.normal(0, sigma, N)
16 F = -k * x + noise
17
18 # Split the data
19 x_train, x_test, F_train, F_test = train_test_split(x, F, test_size=0.1
    , random_state=42, shuffle=True)
20
21 # Linear model (Degree 2)
22 poly = PolynomialFeatures(degree=2)
23 x_train_poly = poly.fit_transform(x_train.reshape(-1, 1))
24 x_test_poly = poly.transform(x_test.reshape(-1, 1))
25
26 model = LinearRegression()
27 model.fit(x_train_poly, F_train)
28
29 # Predictions
30 F_train_pred = model.predict(x_train_poly)
31 F_test_pred = model.predict(x_test_poly)
32
33 # Calculate errors
34 E_in = mean_squared_error(F_train, F_train_pred)
35 E_out = mean_squared_error(F_test, F_test_pred)
36
37 # Calculus of the R^2 error .score() R^2 de 0 a 1
38 r2_train = model.score(x_train_poly, F_train)
39 r2_test = model.score(x_test_poly, F_test)
40
41 print(f"E_in: {E_in}")
42 print(f"E_out: {E_out}")
43 print(f"Precision R^2 Train: {r2_train*100:.2f}%")
44 print(f"Precision R^2 Test: {r2_test*100:.2f}%")
45
46 # Plot data
47 plt.scatter(x_train, F_train, label="Training data", color="blue", alpha
    =0.7)
48 plt.scatter(x_test, F_test, label="Test data", color="green", alpha=0.7)
49
50 x_range_plot = np.linspace(0, 10, 100).reshape(-1, 1)
51 plt.plot(x_range_plot, model.predict(poly.transform(x_range_plot)),
52     label=f"Fitted model (R^2 = {r2_test:.4f})",
53     color="red")
54 plt.xlabel("Displacement (x)")
55 plt.ylabel("Force (F)")
56 plt.title("Model fit visualization")
57 plt.grid(True, alpha=0.3)
58 plt.legend()
59 plt.show()
60
61 # Comparativa de 3 grados
62 plt.figure(figsize=(10, 6))
```

```
63 plt.scatter(x_train, F_train, label="Training data", color="blue", alpha
    =0.5)
64 plt.scatter(x_test, F_test, label="Test data", color="green", alpha=0.7)
65
66 degrees = [1, 2, 3]
67 colors = ['red', 'orange', 'purple']
68
69 for d, c in zip(degrees, colors):
70     poly = PolynomialFeatures(degree=d)
71     x_train_poly = poly.fit_transform(x_train.reshape(-1, 1))
72     x_test_poly = poly.transform(x_test.reshape(-1, 1))
73
74     model = LinearRegression()
75     model.fit(x_train_poly, F_train)
76
77     # R^2 calculus (0 to 1)
78     r2_test = model.score(x_test_poly, F_test)
79     E_out = mean_squared_error(F_test, model.predict(x_test_poly))
80
81     print(f"Grado {d} -> R^2 Test: {r2_test:.4f}, MSE: {E_out:.2f}")
82
83     x_range = np.linspace(0, 10, 100).reshape(-1, 1)
84     y_plot = model.predict(poly.transform(x_range))
85     plt.plot(x_range, y_plot, label=f"Degree {d} (R^2 = {r2_test:.3f})",
        color=c, linewidth=2)
86
87 plt.xlabel("Displacement (x)")
88 plt.ylabel("Force (F)")
89 plt.title("Model Fit Comparison: Polynomial Degrees 1, 2, and 3")
90 plt.grid(True, alpha=0.3)
91 plt.legend()
92 plt.show()
```

Please note that AI has been implemented in order to maximize efficiency on the second part of the code regarding the different degrees plotted together.